

Business Galore

1st Julian David Celis Giraldo
Enginner System Department
University Francisco José de Caldas
Bogotá D.C, Colombia
20222020041

2nd Catalina Ariza Ardila
Enginner System Department
University Francisco José de Caldas
Bogotá D.C, Colombia
20222020084

Abstract—Scheduling appointments for stores over the phone can often be exhausting or frustrating. That’s why we decided to base our own scheduling application on ”Beunik” (an appointment scheduling app for hair salons), but with the difference that we expand the variety of businesses where you can book quickly and at your convenience. As a user, you will be able to either list your own business in the app or simply focus on scheduling appointments.

Index Terms—business software, develop, patterns, software principles

I. INTRODUCTION

An appointment scheduling application can be very useful in terms of speed, as you don’t need to wait for a store associate to answer your call. Additionally, you can calmly view all the available options that suit your needs or preferences. An example of this is the mobile app ”Beunik: Reserve at Beauty Salons and Barbershops,” where you can essentially search for different beauty and barber businesses, schedule your appointment with the employee you choose from the selected store, and select the type of service you require. However, these are not the only benefits that an appointment scheduling app offers. For store owners, it is useful to promote their business on an app and manage appointment information more organized and efficiently.

But why limit ourselves to an application based solely on beauty salons and barbershops? The idea behind our app is that people with any business offering appointment services can add it to the app, allowing users to schedule appointments across a wide variety of businesses.

To build this application, it is essential to understand what the user truly wants to do with it. Therefore, we created user stories based on our criteria to gain clarity and make the application as accurate as possible. Additionally, we utilize the object-oriented paradigm, which allows us to represent a real-world idea or problem in software, among other benefits.

The objective of this application is not only for users to add their business or schedule an appointment but also to provide a flexible application that can easily adapt to new user requirements. Furthermore, by implementing the object-oriented paradigm, we can work modularly, ensuring that maintenance is straightforward.

II. METHOD AND MATERIALS

A. Prior to software development

Using the object-oriented paradigm, we started creating CRC cards to define what classes our program will have and what their responsibilities and collaborators are, with the help of the information provided by the user stories.

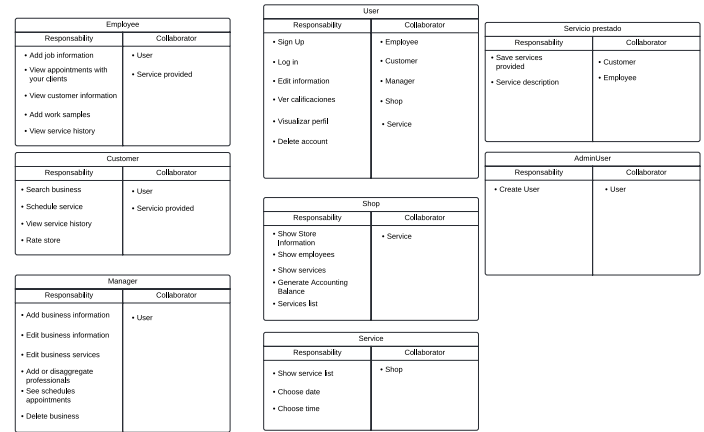


Fig. 1. CRC Cards.

From this, we began constructing class diagrams, activity diagrams, deployment diagrams, and sequence diagrams to gain a more detailed view of the application’s functionality.

By the other hand, to ensure better organization and a clear division of responsibilities, we structured the project into three main modules: user module, reservation module, and store module. This separation allowed us to define precisely which programming language to use for each component.

Java was used to develop the user module and the store module. Python was used for managing reservations. Additionally, we applied best programming practices, implemented design patterns, avoided code smells, and followed a layered and microservices-based architecture to enhance scalability and maintainability.

To enable communication between modules, we utilized APIs, facilitating efficient and modular interaction among the different system components. Finally, we implemented Docker to efficiently deploy and manage services.

III. RESULTS

We have defined the user stories, limited the scope of our application, created diagrams, and generally documented our project, thus providing a blueprint of what we will develop in code. Also, as a result, we developed a clean, flexible, and robust application, where modules communicate through APIs, ensuring seamless integration of functionalities. This allowed us to meet multiple user requirements, guaranteeing a scalable and easy-to-maintain system.

IV. CONCLUSIONS

Throughout the development of this document, the importance of design patterns, software architecture, and best programming practices has been evident. When properly implemented, these elements contribute to a more maintainable, robust, flexible, and clean codebase, among other benefits.

The use of UML diagrams, user stories with acceptance criteria, and CRC cards has streamlined code construction, making the development process faster and more structured. These tools have helped ensure that the final product aligns closely with the needs and expectations of the end user. By leveraging APIs, we were able to develop our project across different programming environments, allowing us to work more efficiently while taking advantage of the best features each environment has to offer.

One of the most critical aspects of application development is organization, especially in team-based projects. Implementing an agile methodology significantly improved our workflow and collaboration throughout the project. Applying SOLID principles in object-oriented projects is fundamental to achieving a well-structured and scalable program.

Additionally, the use of Docker simplified the creation, deployment, and management of our application through containerization. This was particularly useful in our case, where multiple backends needed to interact seamlessly. Docker's scalability also makes it an ideal choice for microservices-based applications like ours.

REFERENCES

- [1] [1] Beunik. "Beunik: Reserve in Beauty Salons and Barbershops." [Online]. Available: <https://beunik.co/>. [Accessed: Oct. 3, 2024].
- [2] C. A. Martínez, "Características de la Programación Orientada a Objetos," *Cibernética 1*. [Online]. Available: <https://portalacademico.cch.unam.mx/cibernetica1/algoritmos-y-codificacion/caracteristicas-POO> [Accessed: Oct. 3, 2024].